

PROJECT REPORT

On

AUTOVERSE

A Car Discovery, Recommendation and Ownership Cost Platform for the Indian Market

Submitted by

Karelia Malav H (IU2341230017)

In fulfillment for the award of the degree

Of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING



INSTITUTE OF TECHNOLOGY AND ENGINEERING

INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ

AHMEDABAD-382115, GUJARAT, INDIA

WEB: www.indusuni.ac.in

SEPTEMBER 2026

PROJECT REPORT

ON

AUTOVERSE

AT



In the partial fulfillment of the requirement

for the degree of

Bachelor of Technology

in

Computer Science And Engineering

PREPARED BY

Karelia Malav H (IU2341230017)

UNDER GUIDANCE OF

Internal Guide

Mr. Hirenkumar Mer

Assistant Professor

Department of Computer Science And Engineering, Indus Institute of Technology and
Engineering

INDUS UNIVERSITY – Ahmedabad, State: Gujarat

SUBMITTED TO
INSTITUTE OF TECHNOLOGY AND ENGINEERING
INDUS UNIVERSITY CAMPUS, RANCHARDA, VIA-THALTEJ
AHMEDABAD-382115, GUJARAT, INDIA,

WEB: www.indusuni.ac.in

SEPTEMBER 2026

INDUS INSTITUTE OF TECHNOLOGY AND ENGINEERING

COMPUTER SCIENCE AND ENGINEERING

2026 – 2027



CERTIFICATE

Date: 24/09/2026

This is to certify that the project work entitled "AUTOVERSE" has been carried out by KARELIA MALAV H under my guidance in partial fulfillment of the degree of Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING (Final Year) of Indus University, Ahmedabad during the academic year 2026-2027.

Mr. Hirenkumar Mer
Assistant Professor,
Department of Computer Science
And Engineering, IITE, Indus
University, Ahmedabad

Dr. Kaushal Jani
Head of Department,
Department of Computer Science
And Engineering, IITE, Indus
University, Ahmedabad

Prof. Zalak Vyas
Head of Department,
Department of Computer Science
And Engineering, IITE, Indus
University, Ahmedabad

CANDIDATE'S DECLARATION

I declare that the final year project report entitled "AutoVerse" is my own work, carried out entirely by me under the supervision of the guide Mr. Hirenkumar Mer.

I further declare that, to the best of my knowledge, this report does not contain any part of work that has been submitted for the award of any degree, in this university or any other university, without proper citation.

Candidate's Signature

Karelia Malav H (IU2341230017)

Guide: Mr. Hirenkumar Mer

Assistant Professor

Department of Computer Science And Engineering, Indus Institute of Technology and Engineering

INDUS UNIVERSITY – Ahmedabad, State: Gujarat

ACKNOWLEDGEMENT

Towards the successful completion of my B.Tech in Computer Science and Engineering final year project at Indus University, I would like to express my sincere gratitude to everyone who supported me, guided me, and encouraged me throughout the course of this project.

I am deeply thankful to my guide, Mr. Hirenkumar Mer, Assistant Professor, Department of Computer Science and Engineering, for his constant guidance, patience and encouragement throughout the development of AutoVerse. His feedback at every stage, from finalizing the project scope to reviewing the system design and the final report, helped me turn an initial idea into a complete, working platform.

I would also like to sincerely thank Dr. Kaushal Jani and Prof. Zalak Vyas, Heads of the Department of Computer Science and Engineering, and the rest of the faculty at Indus Institute of Technology and Engineering, for creating an environment that encouraged independent, practical project work, and for their support throughout this final year.

On a personal note, I would like to thank my family and friends for their patience, encouragement and support during the many hours spent building, testing and documenting this project. Their support made it much easier to stay consistent and see the project through to completion.

- Karelia Malav H
IU2341230017
Computer Science & Engineering

TABLE OF CONTENT

Title	Page No
ABSTRACT.....	10
LIST OF FIGURES.....	11
LIST OF TABLES	11
ABBREVIATIONS.....	11
CHAPTER 1 INTRODUCTION	13
1.1 Project Summary	13
1.2 Project Purpose	13
1.3 Project Scope	13
1.4 Objectives	14
1.4.1 Main Objectives	14
1.4.2 Secondary Objectives	14
1.5 Technology and Literature Overview	14
1.5.1 Frontend Technology.....	14
1.5.2 Backend Technology.....	14
1.5.3 Programming Language.....	14
1.5.4 Libraries and Tools.....	14
1.5.5 Literature Overview.....	15
1.6 Synopsis	15
CHAPTER 2 LITERATURE SURVEY	17
2.1 Introduction of Survey.....	17
2.2 Why Survey?	17
2.3 Existing System VS Proposed System	17
CHAPTER 3 PROJECT MANAGEMENT.....	19
3.1 Project Planning Objectives.....	19
3.2 Project Scheduling	19
3.2.1 Basic Principle Scope	19
3.2.2 Compartmentalization Scope	19

3.2.3 Work Breakdown Structure	19
3.2.4 Project Organization	20
3.2.5 Timeline Chart	20
3.2.5.1 Time Allocation	20
3.3 Risk Management	20
CHAPTER 4 SYSTEM REQUIREMENTS.....	22
4.1 User Characteristics.....	22
4.2 Functional Requirement	22
4.3 Activity and Proposed System	22
4.4 Non Functional Requirement	22
4.5 Hardware and Software Requirements.....	23
CHAPTER 5 SYSTEM ANALYSIS	25
5.1 Study of Current System.....	25
5.2 Problems in Current System	25
5.3 Requirement of new System	25
5.4 Process Model	25
5.5 Feasibility Study.....	25
5.5.1 Technical Feasibility.....	25
5.5.2 Operational Feasibility.....	26
5.5.3 Economical Feasibility	26
5.5.4 Schedule Feasibility	26
5.6 Features of New System.....	26
CHAPTER 6 Testing	28
6.1 Black-Box Testing.....	28
6.2 White-Box Testing	28
6.3 Test Cases	28
CHAPTER 7 SYSTEM DESIGN.....	31
7.1 Class Diagram	31
7.2 Use-Case Diagram.....	32
7.3 Sequence Diagram.....	33
7.4 Activity Diagram	34
7.5 Data Flow Diagram (DFD)	35

7.5.1 Level 0 DFD	35
7.5.2 Level 1 DFD	36
CHAPTER 8 RESULT AND DISCUSSION	38
8.1 Home Page / Browse Cars	38
8.2 Car Detail Page	38
8.3 Compare & Recommend	39
8.4 Ownership Cost Calculator	40
8.5 Favorites	40
8.6 Overall System Performance	41
CHAPTER 9 LIMITATION AND FUTURE ENHANCEMENT	43
9.1 Limitation.....	43
9.2 Future Enhancement.....	43
CHAPTER 10 CONCLUSION	45
10.1 Conclusion	45
CHAPTER 11 Appendices.....	47
11.1 Business Model.....	47
11.2 Product Deployment Detail	47
11.3 API and Web Service Details.....	47
BIBLIOGRAPHY.....	48

ABSTRACT

AutoVerse is a full-stack web platform that helps people in India search for, compare and understand cars before buying one. Car buyers today have to depend on several different websites to get a full picture of a car - one site for specifications, another for on-road price, a third for user reviews - and none of them show what the car will actually cost to own over the next five or ten years. AutoVerse brings all of this into a single place.

The platform is built around a searchable catalog of more than 10,000 cars spanning over 20 major Indian brands, with filters for budget, body type, fuel type, seating capacity and brand. A user can browse the catalog without creating an account, open any car's detail page to see its full specification sheet, and add cars to a side-by-side comparison view that lines up specifications, pricing and features across multiple models at once.

A recommendation engine ranks cars against a buyer's stated preferences - budget, fuel type, mileage and body type - and returns a shortlist instead of leaving the buyer to manually filter through thousands of listings. A year-by-year ownership cost calculator then estimates what a chosen car will actually cost to keep, combining fuel or charging cost, insurance, service and depreciation using the schedule published by Indian insurance regulator IRDAI, so the estimate is grounded in a real, published depreciation curve rather than a rough guess.

The system is built with a React frontend, a FastAPI backend, and a PostgreSQL database, and is deployed live across three separate platforms - the frontend on Vercel, the backend API on Render, and the database on Neon - so the finished project is a working, publicly reachable application rather than something that only runs locally on one machine.

In conclusion, AutoVerse demonstrates that car search, side-by-side comparison, preference-based recommendation and long-term ownership cost estimation can be combined into a single, coherent platform. It closes the specific gap identified during the literature survey in Chapter 2 - that no single existing Indian car-shopping website offers all four of these features together - and does so as a complete, deployed, end-to-end system built over one academic semester.

LIST OF FIGURES

Figure No	Title
Figure 7.1	Class Diagram
Figure 7.2	Use-Case Diagram
Figure 7.3	Sequence Diagram
Figure 7.4	Activity Diagram
Figure 7.5.1	DFD Level 0
Figure 7.5.2	DFD Level 1
Figure 8.1	Home Page / Browse Cars
Figure 8.2	Car Detail Page
Figure 8.3a	Compare Cars
Figure 8.3b	Recommendations
Figure 8.4	Ownership Cost Calculator
Figure 8.5	Favorites

LIST OF TABLES

Table No	Title
Table 2.3	Existing Systems vs AutoVerse
Table 3.2.5.1	Time Allocation
Table 3.3	Risk Management

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
UI	User Interface
UX	User Experience
JWT	JSON Web Token
HTTP	HyperText Transfer Protocol
REST	Representational State Transfer
SQL	Structured Query Language
ORM	Object Relational Mapping
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
EMI	Equated Monthly Installment
IRDAI	Insurance Regulatory and Development Authority of India
DFD	Data Flow Diagram
SRS	Software Requirements Specification

CHAPTER 1

INTRODUCTION

- **PROJECT SUMMARY**
- **PROJECT PURPOSE**
- **PROJECT SCOPE**
- **OBJECTIVES**
- **TECHNOLOGY AND LITERATURE
OVERVIEW**
- **SYNOPSIS**

CHAPTER 1 INTRODUCTION

1.1 Project Summary

AutoVerse is a web application built to make car buying easier for people in India. Instead of checking one website for specifications, another for on-road price, and a third for resale value, AutoVerse brings all of this together in a single platform.

A user can search and filter a catalog of over 10,000 cars, look at full specifications for any car, compare two or three cars side by side, and get a ranked list of cars that fit their budget and preferences. The platform also includes a cost calculator that shows what a car will actually cost to own over several years — fuel, insurance, servicing and the value it will lose — instead of showing only the price at the time of purchase.

The application is built using React for the frontend, FastAPI (Python) for the backend, and PostgreSQL for the database. It is deployed and publicly accessible, using Vercel to host the frontend, Render to host the backend, and Neon to host the database.

1.2 Project Purpose

The purpose of AutoVerse is to give car buyers one reliable place to research a car before spending a large amount of money on it. Buying a car is one of the biggest purchases most Indian households make, and the information needed to make that decision responsibly is often scattered across many different websites.

AutoVerse aims to close three specific gaps found in existing platforms: cost information is usually shown as a single number instead of a year-by-year breakdown; filters exist but nothing ranks cars against what a specific buyer actually needs; and resale value is rarely calculated using the same depreciation schedule that Indian insurers use.

1.3 Project Scope

The system is capable of:

- Searching and filtering the full car catalog by brand, fuel type, body type, transmission, price and seating capacity.
- Showing full specifications for any selected car.
- Comparing two or three shortlisted cars side by side.
- Generating ranked recommendations from a buyer's budget, fuel type and mileage preferences.
- Calculating year-wise ownership cost — fuel, insurance, maintenance and depreciation.
- Showing a depreciation schedule aligned to the IRDAI standard, adjusted for the car's condition.
- Letting a registered user save cars to a Favorites list and view their recently viewed cars.
- Providing a secure login and password-reset flow.

The current version has a few known limits:

- It is a web application only — there is no dedicated mobile app yet.
- It does not connect to a payment gateway or handle actual purchases.
- Fuel price is entered manually by the user rather than fetched automatically.
- Recommendations are based on a fixed scoring formula, not on machine learning.

1.4 Objectives

1.4.1 Main Objectives

- To build a single platform for car search, comparison and evaluation.
- To rank cars against a buyer's own budget and preferences, not just filter them.
- To show ownership cost broken down by year, not as one guessed number.
- To keep the depreciation numbers aligned to a real, published standard (IRDAI).
- To provide secure user accounts for saving favorite cars.

1.4.2 Secondary Objectives

- To make the interface simple enough for a non-technical buyer to use without help.
- To make the platform work well on both desktop and mobile screens.
- To keep the codebase organized so new features can be added later without rewriting old ones.
- To document the API clearly enough that another developer could use it.

1.5 Technology and Literature Overview

1.5.1 Frontend Technology

The interface is built with React (a JavaScript library for building interactive web pages) and Vite (a fast development and build tool), styled using Tailwind CSS, and communicates with the backend using Axios.

1.5.2 Backend Technology

The backend is written in Python using FastAPI, a modern framework that automatically validates incoming data and generates interactive API documentation. Database access uses SQLAlchemy, and Pydantic checks that every request and response matches the expected shape before it is processed. Car, brand, user and favorite records are stored in PostgreSQL, hosted on Neon for the live deployment.

1.5.3 Programming Language

Programming languages are used to develop software systems by providing instructions a computer can execute. AutoVerse's backend and data pipeline are written in Python, and its frontend is written in JavaScript (as React/JSX).

1.5.4 Libraries and Tools

The project uses several important libraries:

- React.js — used for building the interactive user interface.

- Vite — provides fast builds and a live development server.
- SQLAlchemy — used for interacting with the PostgreSQL database and defining its schema.
- Axios — used for making API requests between the frontend and backend.

1.5.5 Literature Overview

Web-based platforms for comparing and buying vehicles have grown quickly in recent years, since they let a buyer research a purchase without visiting a showroom first. Existing Indian platforms such as CarDekho, CarWale and Cars24 offer large catalogs and comparison tools, but none of them combine search, comparison, personalized ranking and a transparent, standard-based cost projection in one place — the specific gap AutoVerse is built to close. This is expanded on in Chapter 2.

1.6 Synopsis

AutoVerse is a web-based car discovery and evaluation platform aimed at Indian car buyers. It replaces the fragmented experience of visiting several sites with one platform that covers search, comparison, personalized recommendations, and a detailed ownership cost calculator. It is built with React, FastAPI and PostgreSQL, and is already deployed and live for real use.

CHAPTER 2

LITERATURE SURVEY

- **INTRODUCTION OF SURVEY**
- **WHY SURVEY?**
- **EXISTING SYSTEM VS PROPOSED SYSTEM**

CHAPTER 2 LITERATURE SURVEY

2.1 Introduction of Survey

Before building AutoVerse, existing car-shopping websites available to Indian buyers were studied to understand what they already do well, and where they fall short. This helped shape which features AutoVerse should focus on.

2.2 Why Survey?

A survey of existing platforms is useful because it shows what buyers can already do elsewhere, so a new project does not simply repeat what already exists. It also highlights real gaps that a new system can be built specifically to close, instead of guessing what buyers might need.

Three widely used Indian car-shopping platforms were reviewed: CarDekho, CarWale and Cars24.

2.3 Existing System VS Proposed System

Sr. No.	Aspect	Existing Systems	AutoVerse (Proposed)
1	Cost Information	Shown as a single on-road price	Broken down year by year: fuel, insurance, maintenance, depreciation
2	Comparison	Spec-only, side by side	Spec comparison plus a scored, ranked recommendation
3	Personalization	Filters only narrow the list	Cars are scored and ranked against the buyer's own budget and preferences
4	Depreciation	Rarely tied to a published standard	Follows the IRDAI schedule used by Indian insurers
5	Platform Coverage	Search, pricing and valuation split across different sites	Search, comparison, recommendation and cost calculation on one platform
6	API Access	Not publicly documented	Open, documented REST API

Table 2.3: Existing System VS Proposed System

Across all three platforms studied, the same three gaps kept showing up:

- Cost information is fragmented — on-road price, insurance and resale value are shown separately, never as one combined, year-by-year figure.
- Filtering exists everywhere, but no platform scores or ranks results against a specific buyer's budget, fuel type and usage.
- Where depreciation is shown at all, it is rarely tied to a standard, insurer-recognized schedule such as the one published by IRDAI.

AutoVerse is built directly around closing these three gaps.

CHAPTER 3

PROJECT MANAGEMENT

- **PROJECT PLANNING OBJECTIVES**
- **PROJECT SCHEDULING**
- **RISK MANAGEMENT**

CHAPTER 3 PROJECT MANAGEMENT

3.1 Project Planning Objectives

- Define clear goals — what the platform needs to do, before writing any code.
- Gather requirements — decide what features are needed based on the literature survey.
- Design the system — plan the database, backend and frontend structure before building it.
- Build in stages — the data layer first, then the backend, then the frontend, so each part can be checked before the next is built on top of it.
- Test continuously — verify each feature as it is built rather than only at the end.
- Keep the code organized — so a new feature does not require rewriting old code.
- Deploy the finished platform — make it available on the internet for real use.

3.2 Project Scheduling

3.2.1 Basic Principle Scope

The project was built in layers, one on top of the other: the database first, then the backend API, and finally the frontend that calls that API. Each layer was checked on its own — for example, the backend was tested through its own interactive documentation page before the frontend was connected to it — so problems were caught early, one layer at a time.

3.2.2 Compartmentalization Scope

The system was split into clear, separate modules so each part could be built and tested on its own: a data layer (car and brand records), a backend layer (API routes and business logic — search, comparison, recommendation, cost calculation), and a frontend layer (the pages a user actually sees). Splitting the work this way meant a bug in one module rarely affected the others, and each module could be developed in parallel with the others once its interface was agreed.

3.2.3 Work Breakdown Structure

Phase 1 — Requirement Analysis

- Study existing car-shopping platforms.
- Identify gaps and define what AutoVerse should do.
- Source and clean a 10,000+ record Indian car dataset.

Phase 2 — System Design

- Design the Brand/Car/User/Favorite database schema.
- Plan the API endpoints and page layouts.
- Design the UI/UX for search, comparison, recommendations and the calculator.

Phase 3 — Development

- Build the ingestion pipeline and load the dataset into the database.
- Build the FastAPI backend — routes, validation and business logic.
- Build the React frontend and connect it to the backend.

Phase 4 — Testing

- Test every API endpoint with valid and invalid data.
- Test the UI on desktop and mobile screens.
- Fix bugs found during manual testing.

Phase 5 — Deployment & Maintenance

- Move the database to a cloud host (Neon).
- Deploy the backend (Render) and frontend (Vercel).
- Monitor the live site and fix issues as they come up.

3.2.4 Project Organization

- **Project Guide:** Guides and reviews the project, gives feedback on approach and progress.
- **Developer (Author):** Designs, builds, tests and documents the entire system — frontend, backend and database.
- **Car Buyers (End Users):** The end users of the finished platform: they search, compare, get recommendations and calculate cost.

3.2.5 Timeline Chart

3.2.5.1 Time Allocation

Phase	Duration	Description
Requirement Analysis	2 Weeks	Study existing platforms, source the dataset, define scope.
System Design	2 Weeks	Design database schema, API structure and UI layout.
Development	6 Weeks	Build the ingestion pipeline, backend API and frontend UI.
Testing	2 Weeks	Test API endpoints and UI flows, fix bugs found.
Deployment & Maintenance	2 Weeks	Deploy to Neon, Render and Vercel; monitor and fix live issues.

Table 3.2.5.1: Time Allocation

3.3 Risk Management

Risk	Description	Mitigation Strategy
Technical Issues	Bugs, crashes, or problems connecting different parts of the system.	Regular testing after each feature, using well-supported frameworks.
Data Accuracy	The car dataset could contain errors — wrong prices, mileage, or impossible fuel/transmission combinations.	A validation pipeline checks the dataset for sanity errors before it is used.
Server Downtime	The free hosting tier can go to sleep after inactivity, causing a slow first load.	Documented in Chapter 9 as a known limitation; the user is shown a loading message while the server wakes up.
Scope Creep	Adding too many features could delay finishing the core ones.	Kept a clear in-scope / future-scope split (Section 1.3) from the start.
Time Constraints	Any one phase running long could delay the whole project.	Followed the phase-by-phase schedule in Section 3.2.4 and adjusted priorities when needed.

Table 3.3: Risk Management

CHAPTER 4

SYSTEM REQUIREMENTS

- **USER CHARACTERISTICS**
- **FUNCTIONAL REQUIREMENT**
- **ACTIVITY AND PROPOSED SYSTEM**
- **NON FUNCTIONAL REQUIREMENT**
- **HARDWARE AND SOFTWARE
REQUIREMENTS**

CHAPTER 4 SYSTEM REQUIREMENTS

4.1 User Characteristics

AutoVerse has two kinds of users: guests, who can search, filter, compare cars, get recommendations and use the cost calculator without logging in; and registered users, who can additionally save cars to a Favorites list. No special technical knowledge is needed — the interface is designed to be usable by anyone comfortable browsing a normal website on a phone or computer.

4.2 Functional Requirement

ID	Requirement
FR1	The system shall let users filter cars by brand, fuel type, body type, transmission, price range and seating capacity.
FR2	The system shall display full specifications for any selected car.
FR3	The system shall generate ranked recommendations from the budget, fuel type and mileage preferences a user enters.
FR4	The system shall calculate projected ownership cost from price, ownership duration, annual distance, fuel price and mileage.
FR5	The system shall compute year-wise depreciation using the IRDAI schedule, adjusted by vehicle condition.
FR6	The system shall allow side-by-side comparison of two or three shortlisted cars.
FR7	The system shall let a registered user save and remove cars from a Favorites list.
FR8	The system shall provide secure registration, login and password-reset.
FR9	The system shall expose the above functionality through a documented REST API.

4.3 Activity and Proposed System

A visitor opens AutoVerse and can immediately search or filter the car catalog without logging in. From the results, they can open a car's detail page, add it to a comparison, or run the ownership cost calculator. If they enter budget and preference details, the recommendation engine returns a ranked shortlist. Saving a car to Favorites, or revisiting it later from account history, requires logging in — this is the only part of the flow that is gated behind an account.

4.4 Non Functional Requirement

Category	Requirement
Performance	Filtering across the full 10,000+ record dataset should feel close to instant.
Usability	The interface must work correctly on a phone screen, since most car research realistically starts there.
Reliability	Invalid input (a negative price, an out-of-range year) is rejected before it reaches the database.
Maintainability	Routes, business logic and data models are kept in separate layers, so a new feature does not require touching unrelated code.
Scalability	The data layer supports growth well past 10,000 records without a schema rewrite.
Security	Passwords are hashed, and login uses signed tokens (JWT) rather than storing sessions in plain text.

4.5 Hardware and Software Requirements

Hardware Requirements

As a web application, AutoVerse needs almost no special hardware — any device that can run a modern browser can use it.

- Processor: Any modern dual-core processor or above.
- RAM: 4 GB or more.
- Internet Connection: Required to load the live site and its data.
- Input/Output: A screen and either a keyboard/mouse or a touchscreen.

Software Requirements

Layer	Technology / Version (as used in this project)
Frontend	React 18.3.1, Vite 5.3.3, Tailwind CSS 3.4.6, Axios 1.7.2
Backend	Python 3.11, FastAPI 0.128.8, Uvicorn 0.34.0, SQLAlchemy 2.0.36, Pydantic 2.10.6
Database	PostgreSQL (hosted on Neon)
Hosting	Vercel (frontend), Render (backend), Neon (database)
Tooling	Git & GitHub for version control; FastAPI's built-in Swagger UI (/docs) for API testing

CHAPTER 5

SYSTEM ANALYSIS

- **STUDY OF CURRENT SYSTEM**
- **PROBLEMS IN CURRENT SYSTEM**
- **REQUIREMENT OF NEW SYSTEM**
- **PROCESS MODEL**
- **FEASIBILITY STUDY**
- **FEATURES OF NEW SYSTEM**

CHAPTER 5 SYSTEM ANALYSIS

5.1 Study of Current System

Today, a car buyer researching a purchase in India typically has to visit several different websites: one to compare specifications, another to check on-road pricing, and possibly a third to estimate resale value. None of the popular platforms combine search, comparison, personalized ranking and a transparent cost projection into a single place.

5.2 Problems in Current System

- Fragmented Information — specs, pricing and resale value live on different sites.
- No Personalization — filters narrow a list down, but nothing ranks results against what a specific buyer needs.
- Hidden Long-Term Costs — most sites show only the on-road price, not fuel, insurance, servicing or depreciation.
- Non-Standard Depreciation — where resale value is shown, it rarely follows a recognized standard like the IRDAI schedule.
- Repeated Research — a buyer has to redo the same comparison work across multiple tabs and sites.

5.3 Requirement of new System

- One platform for search, comparison, recommendation and cost calculation.
- A scoring engine that ranks cars against the buyer's actual budget and preferences.
- A year-by-year cost breakdown, not a single guessed number.
- Depreciation figures aligned to the IRDAI standard used by Indian insurers.
- Secure accounts so a user can save cars they are interested in.

5.4 Process Model

AutoVerse was built using an incremental development model: the database layer was built first, followed by the backend API layer, and finally the frontend that consumes that API. Each layer was verified on its own before the next one was built on top of it, which kept problems local and easy to trace back to their source.

5.5 Feasibility Study

5.5.1 Technical Feasibility

AutoVerse uses widely supported, well-documented technologies — React, FastAPI and PostgreSQL — all of which are free and open-source. All three are available as free-tier cloud services (Vercel, Render, Neon), so the project does not need any specialized or costly infrastructure. This makes the system technically feasible to build and to keep running.

5.5.2 Operational Feasibility

The interface is built to be simple enough that a first-time visitor can search, compare and calculate ownership cost without any training or guidance. Most features work without needing an account at all, which keeps the barrier to actually using the platform low.

5.5.3 Economical Feasibility

Because every tool used — the frameworks, the database and the hosting — is free at the scale this project runs at, AutoVerse has no meaningful running cost. This makes it economically feasible to build, deploy and maintain as an academic project.

5.5.4 Schedule Feasibility

The project was broken into clear phases (Section 3.2.4), each with its own time allocation. Building the system in layers — database, then backend, then frontend — meant later phases were not blocked waiting on design decisions still being made, which kept the project on schedule.

5.6 Features of New System

- Multi-filter car search across the full catalog.
- Full specification pages for every car.
- Side-by-side comparison of up to three cars.
- A scoring-based recommendation engine.
- A year-by-year ownership cost calculator.
- An IRDAI-aligned depreciation schedule, adjustable by vehicle condition.
- Secure user accounts with a Favorites list and recently viewed history.
- A responsive interface that works on both desktop and mobile.

CHAPTER 6

TESTING

- **BLACK-BOX TESTING**
- **WHITE-BOX TESTING**
- **TEST CASES**

CHAPTER 6 Testing

6.1 Black-Box Testing

The following black-box tests check that each feature behaves correctly from a user's point of view, without looking at the underlying code:

- User Registration Testing — a new account is created successfully with valid details.
- Invalid Registration Testing — the system rejects a duplicate email or a weak password with a clear message.
- Login Testing — a user can log in with the correct email and password.
- Invalid Login Testing — an incorrect password or unregistered email shows an appropriate error.
- Search & Filter Testing — applying filters returns only cars matching all selected criteria.
- Comparison Testing — selecting up to three cars shows a correct side-by-side comparison.
- Recommendation Testing — submitting budget and preferences returns a ranked, relevant shortlist.
- Ownership Cost Testing — entering ownership duration and distance returns a correct cost breakdown.
- Favorites Testing — a logged-in user can add and remove cars from their Favorites list.
- Password Reset Testing — a reset-link email is sent and successfully updates the password.

6.2 White-Box Testing

The following checks were made against the internal logic of the system:

- Input Validation Logic — Pydantic schemas reject malformed input (negative price, out-of-range year, invalid fuel type) before it reaches the database.
- Recommendation Scoring Logic — the scoring formula (Section 10 equivalent, Chapter 7 design) was manually verified against hand-calculated results for sample cars.
- Ownership Cost Calculation Logic — fuel, insurance, maintenance and depreciation formulas were checked against expected values.
- Database Query Logic — SQLAlchemy queries were checked to confirm correct filtering, joins and pagination.
- Authentication Logic — JWT token generation and password hashing were tested for both valid and invalid credentials.
- API Route Testing — every backend endpoint was exercised through FastAPI's interactive documentation with both valid and invalid payloads.

6.3 Test Cases

Test ID	Test Case	Input	Expected Result
TC1	User Registration	Enter valid name, email and password.	Account is created and the user is logged in.
TC2	Invalid Registration	Leave a required field blank.	A validation error message is shown.
TC3	User Login	Enter a valid email and password.	The user is logged in and redirected to

			the home page.
TC4	Invalid Login	Enter an incorrect password.	"Invalid credentials" message is shown.
TC5	Search Cars	Filter by brand and price range.	Only matching cars are shown.
TC6	Compare Cars	Select three cars and open Compare.	A side-by-side spec table is displayed.
TC7	Get Recommendation	Enter a budget and fuel preference.	A ranked list of matching cars is returned.
TC8	Calculate Ownership Cost	Enter ownership years and annual distance.	A year-wise cost and depreciation breakdown is shown.
TC9	Add to Favorites	Click the favorite icon on a car while logged in.	The car appears in the user's Favorites list.

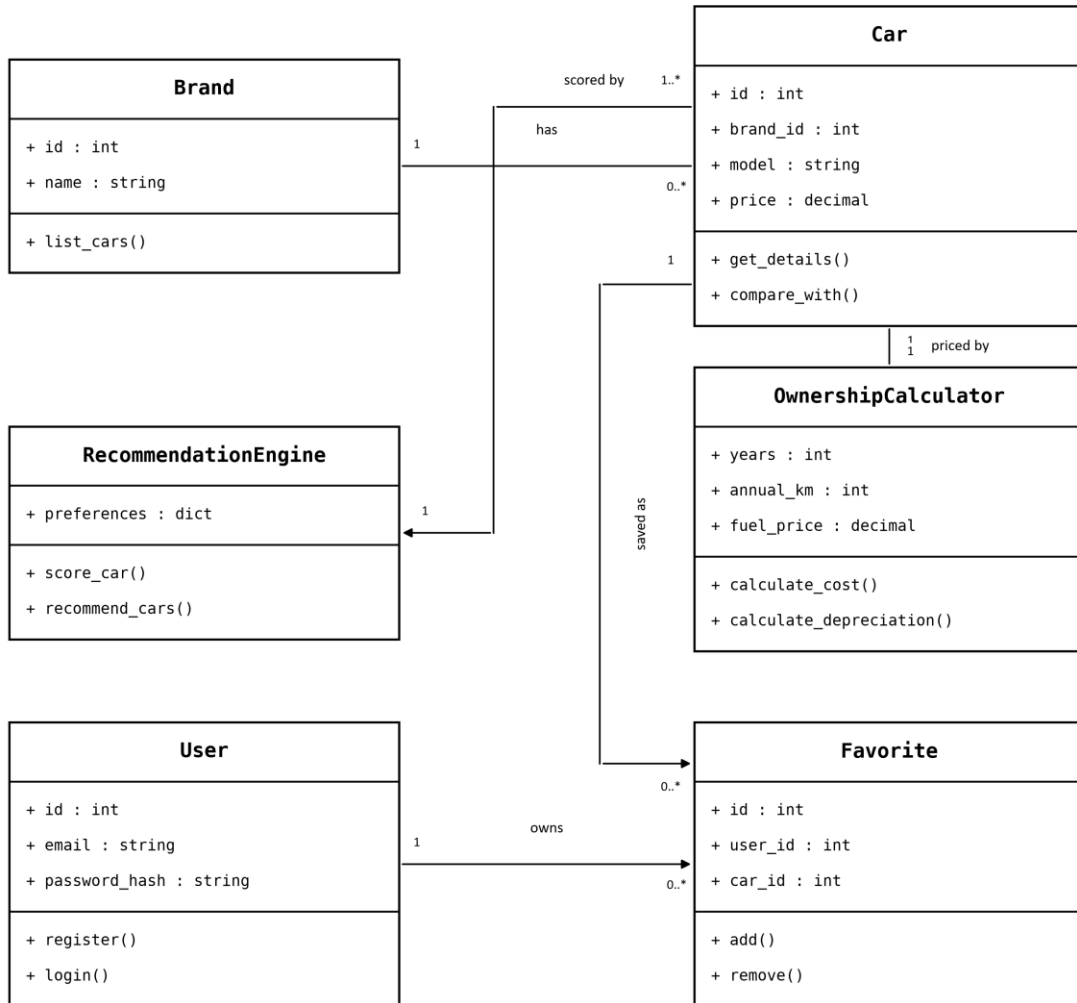
CHAPTER 7

SYSTEM DESIGN

- **CLASS DIAGRAM**
- **USE-CASE DIAGRAM**
- **SEQUENCE DIAGRAM**
- **ACTIVITY DIAGRAM**
- **DATA FLOW DIAGRAM (DFD)**

CHAPTER 7 SYSTEM DESIGN

7.1 Class Diagram



AutoVerse – UML Class Diagram

Figure 7.1: Class Diagram

7.2 Use-Case Diagram

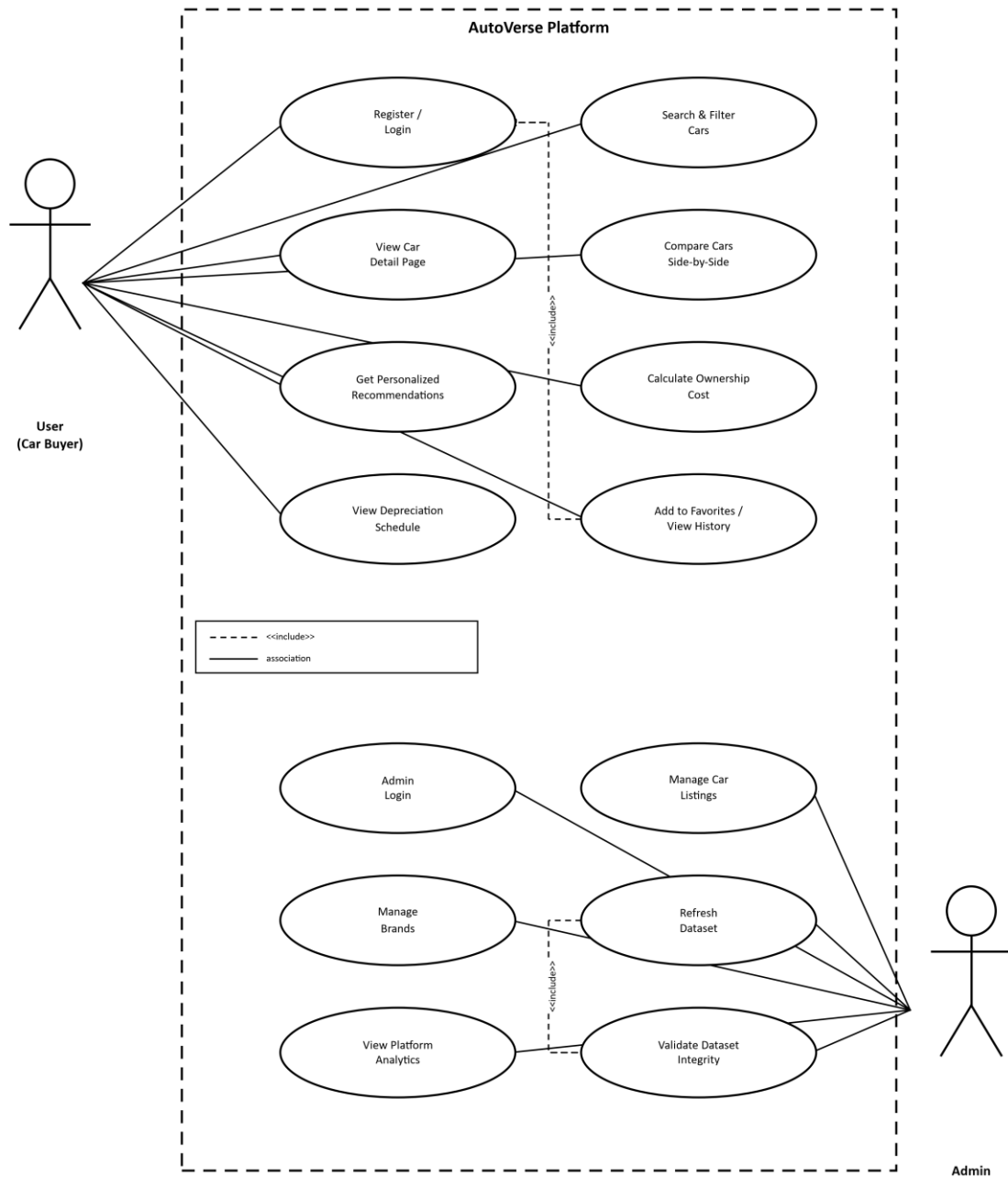


Figure 7.2: Use-Case Diagram

7.3 Sequence Diagram

The sequence diagram below shows one specific interaction — running the Ownership Cost Calculator — step by step, since this flow touches every layer of the system in a single user action.

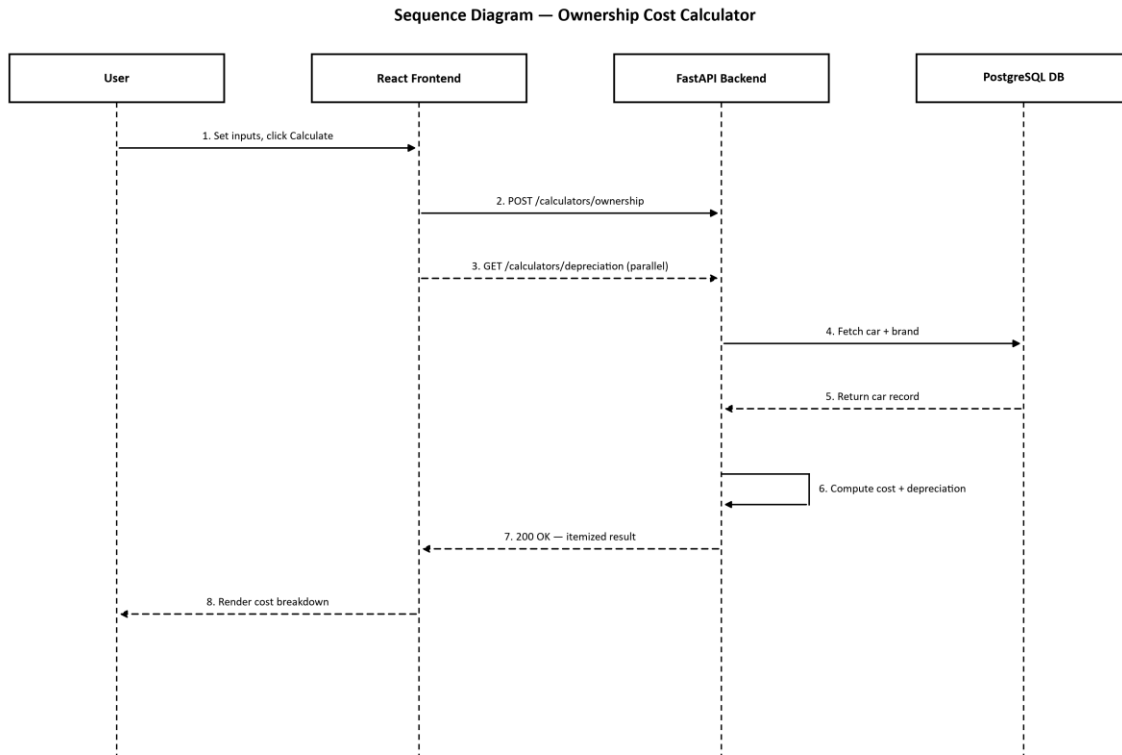


Figure 7.3: Sequence Diagram — Ownership Cost Calculator

7.4 Activity Diagram

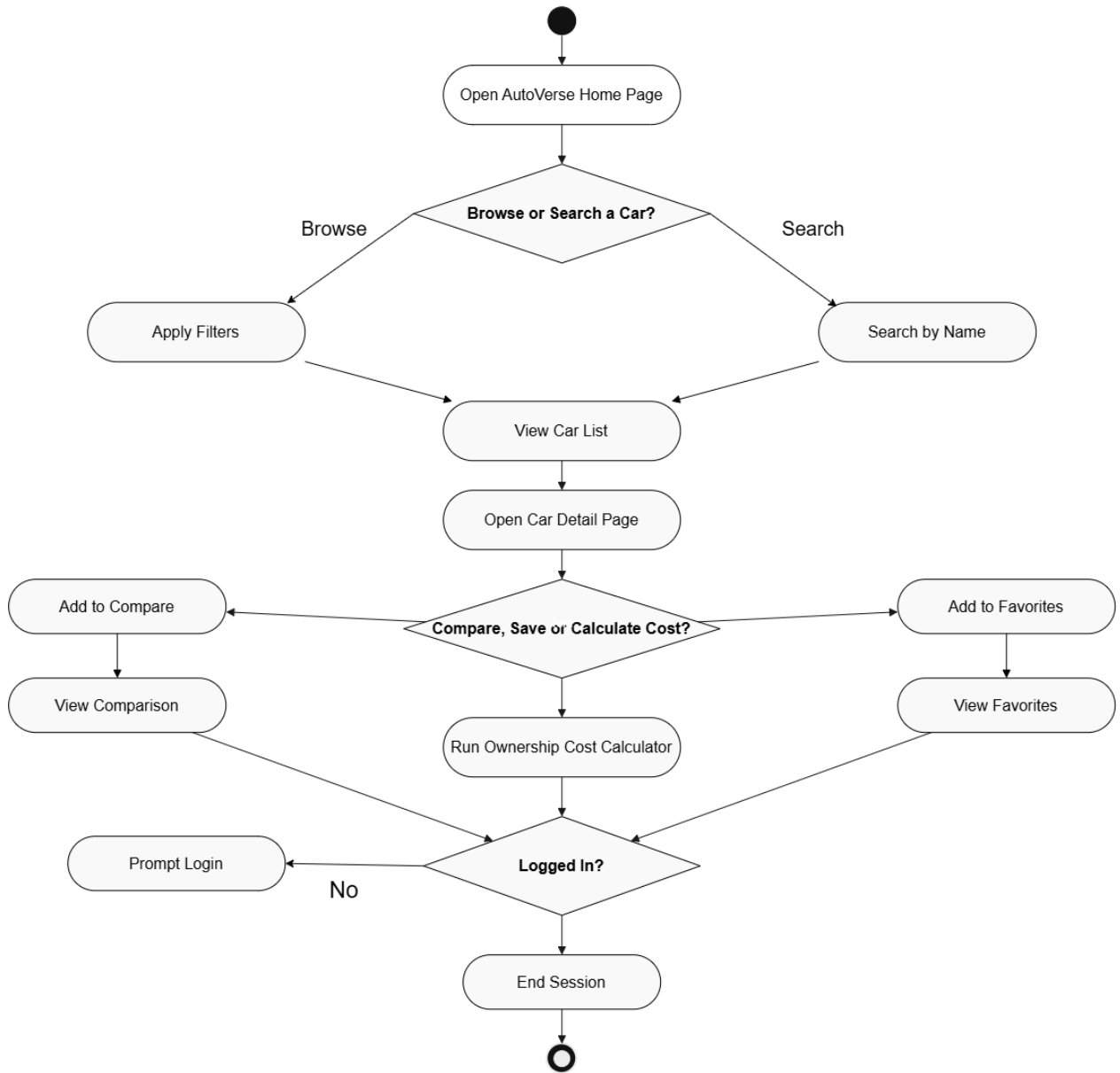


Figure 7.4: Activity Diagram

7.5 Data Flow Diagram (DFD)

7.5.1 Level 0 DFD

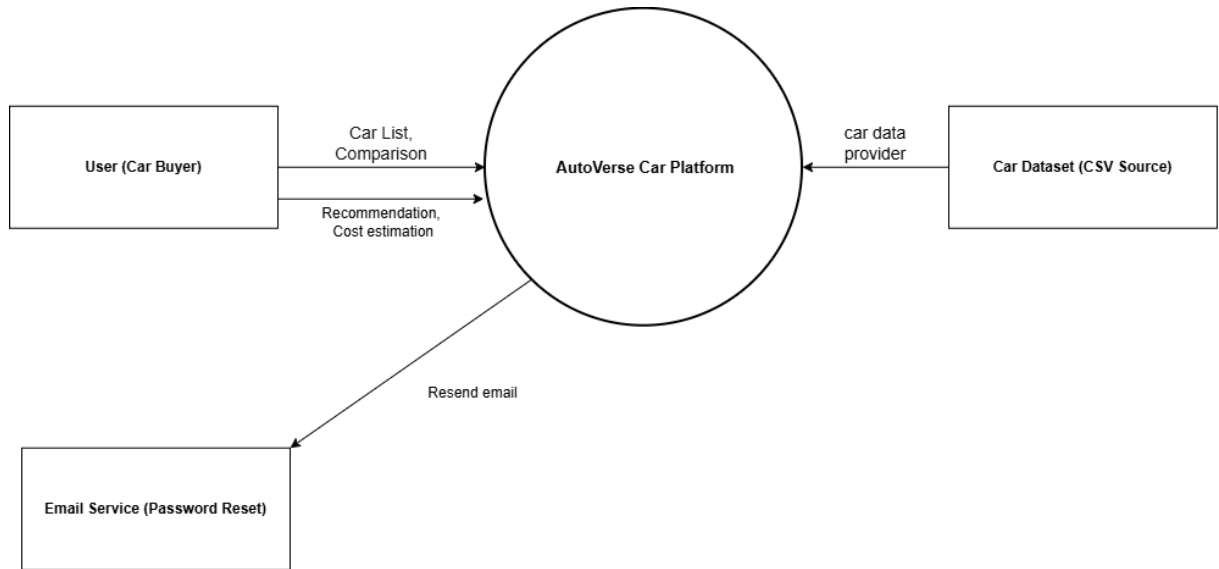


Figure 7.5.1: Level 0 DFD

7.5.2 Level 1 DFD

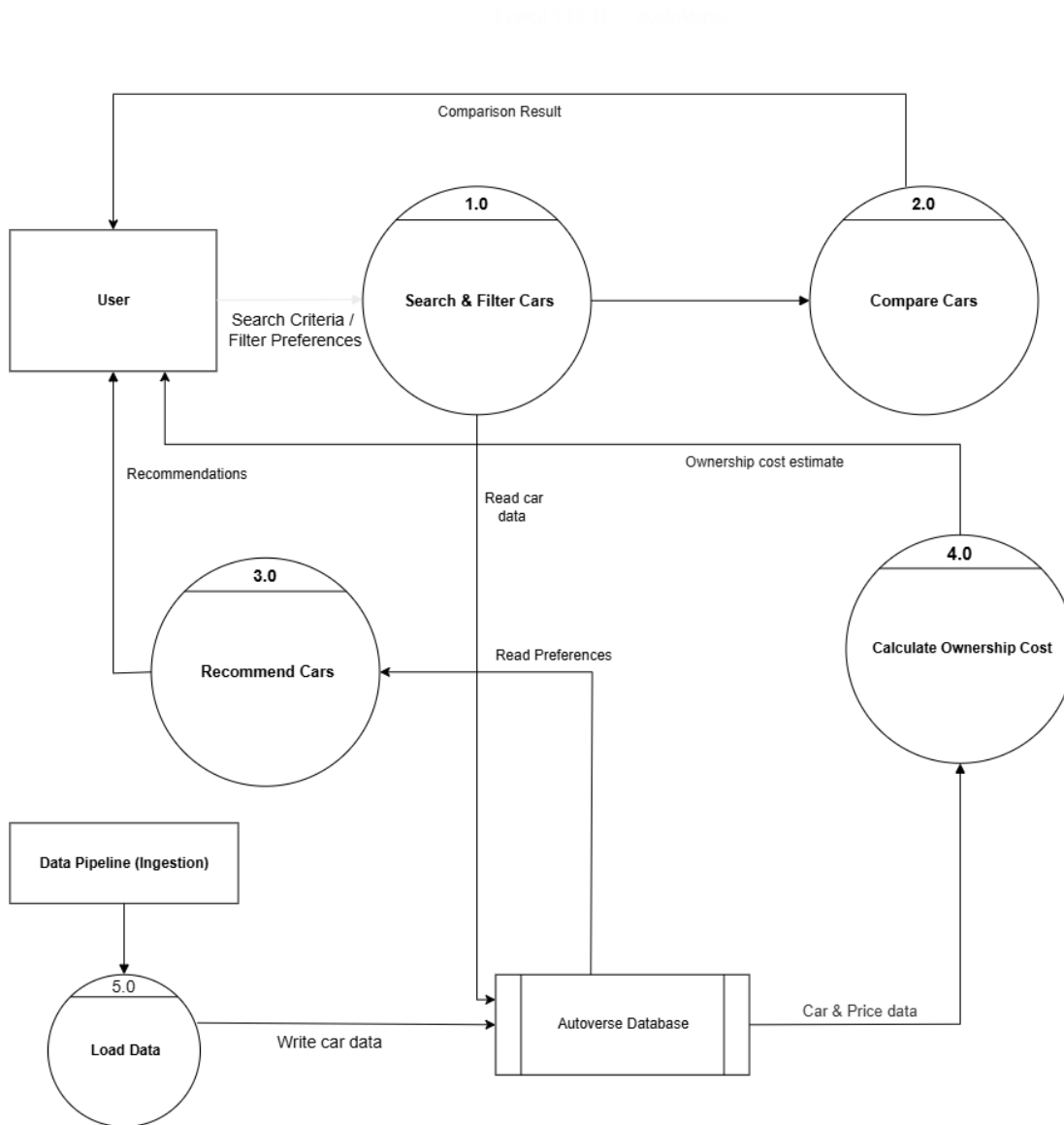


Figure 7.5.2: Level 1 DFD

CHAPTER 8

RESULT AND DISCUSSION

- HOME PAGE / BROWSE CARS
- CAR DETAIL PAGE
- COMPARE & RECOMMEND
- OWNERSHIP COST CALCULATOR
- FAVORITES
- OVERALL SYSTEM PERFORMANCE

CHAPTER 8 RESULT AND DISCUSSION

AutoVerse was built, tested and deployed successfully. The platform is live, and every feature described in the earlier chapters works as designed. This chapter shows the finished screens along with a short discussion of each one.

8.1 Home Page / Browse Cars

The home page shows an overview of the catalog and lets a user jump straight into browsing or searching. The Browse page supports filtering by brand, fuel type, body type, transmission, price range and seating, across all 10,000+ cars.

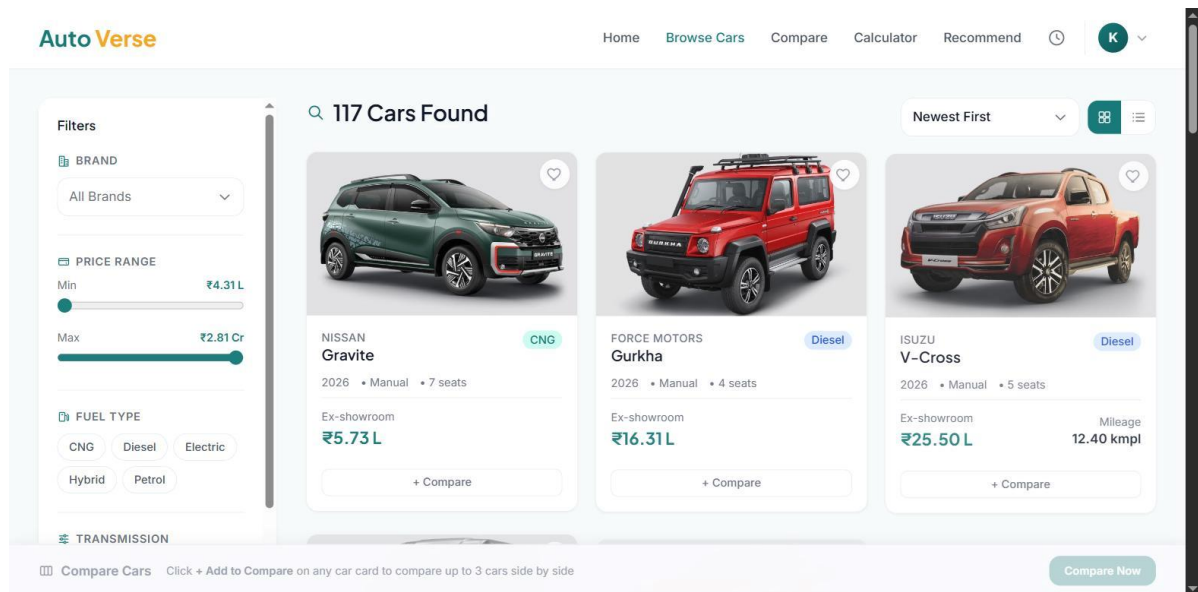


Figure 8.1: Browse Cars

8.2 Car Detail Page

Selecting a car opens its full specification page, along with pricing, images and quick links to compare it or run the cost calculator on it.

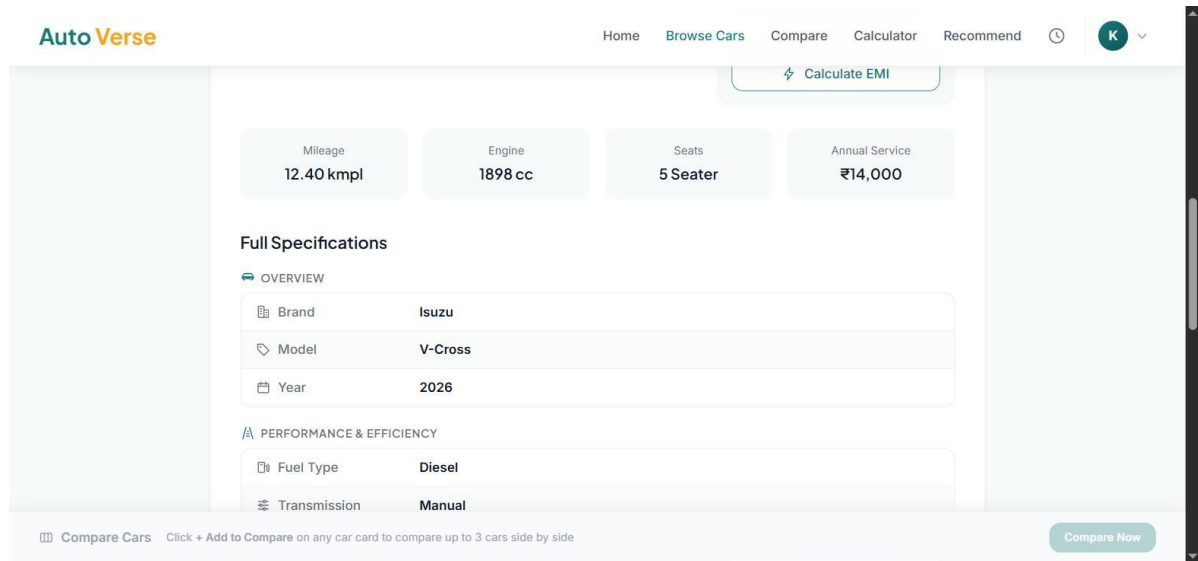


Figure 8.2: Car Detail Page

8.3 Compare & Recommend

Up to three cars can be compared side by side, with the better value in each row highlighted, making it easy to see the real differences between shortlisted options.

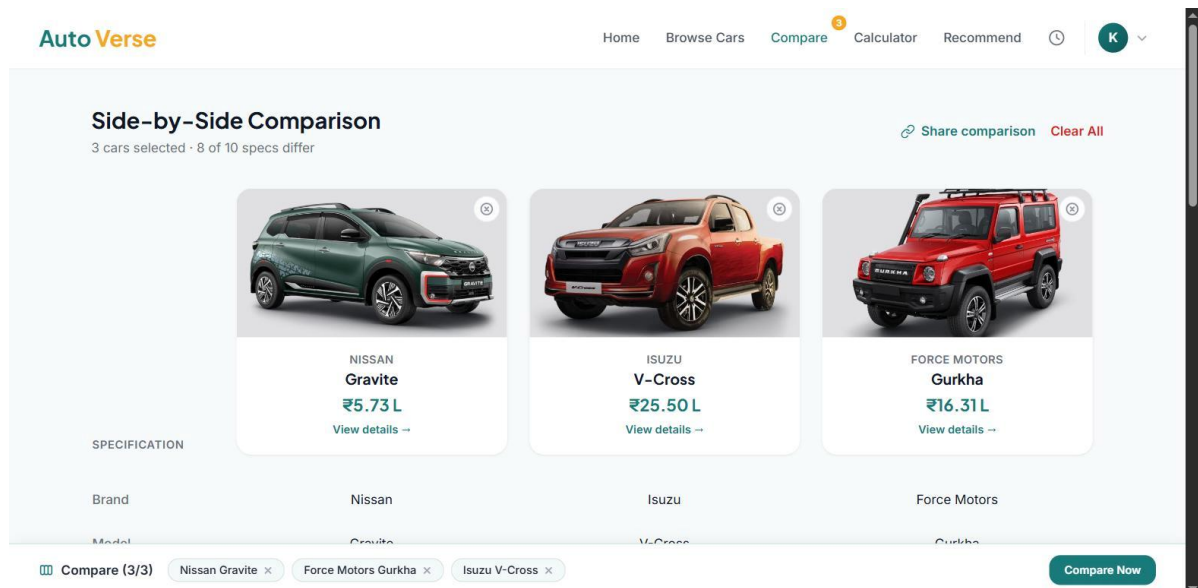


Figure 8.3a: Compare Cars

Entering a budget, fuel preference and mileage requirement returns a ranked list of cars, each with a match score and the specific reasons it was recommended.

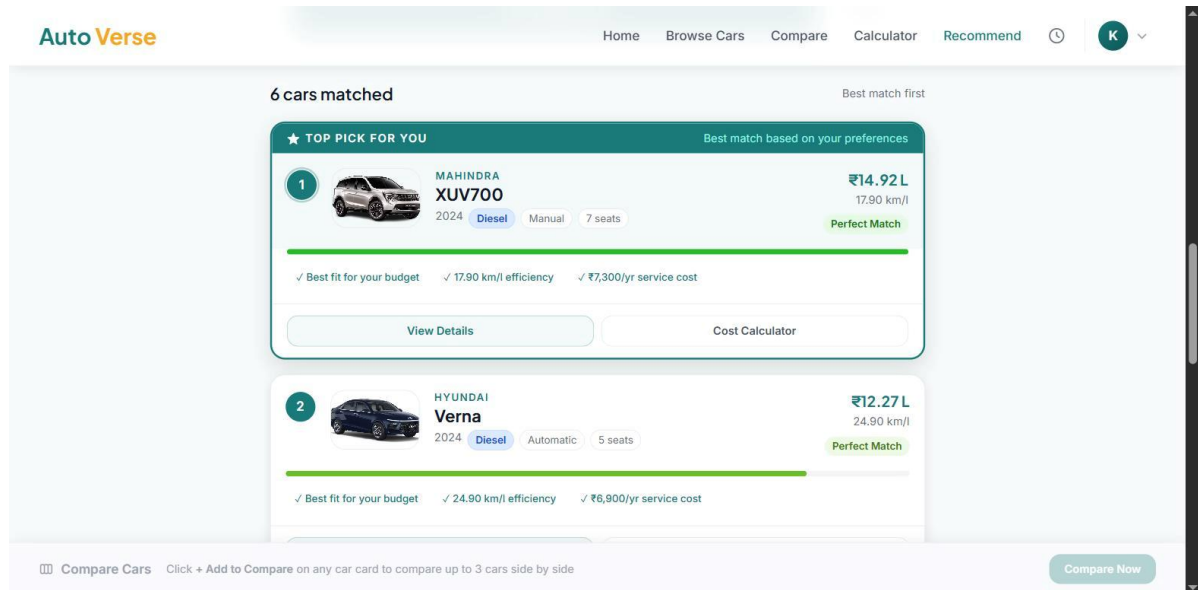


Figure 8.3b: Recommendations

8.4 Ownership Cost Calculator

The calculator projects fuel, insurance, maintenance and depreciation cost across a chosen ownership period, giving a buyer a realistic year-by-year total rather than just the purchase price.

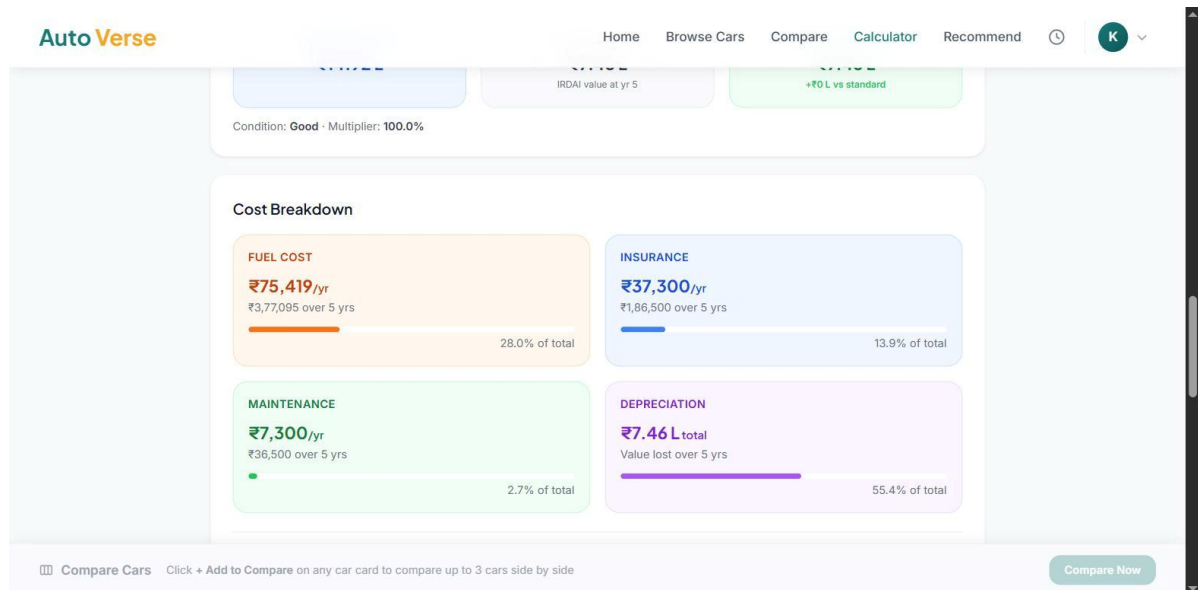


Figure 8.4: Ownership Cost Calculator

8.5 Favorites

A logged-in user can save cars they are interested in to a Favorites list, and return to it anytime without repeating their search.

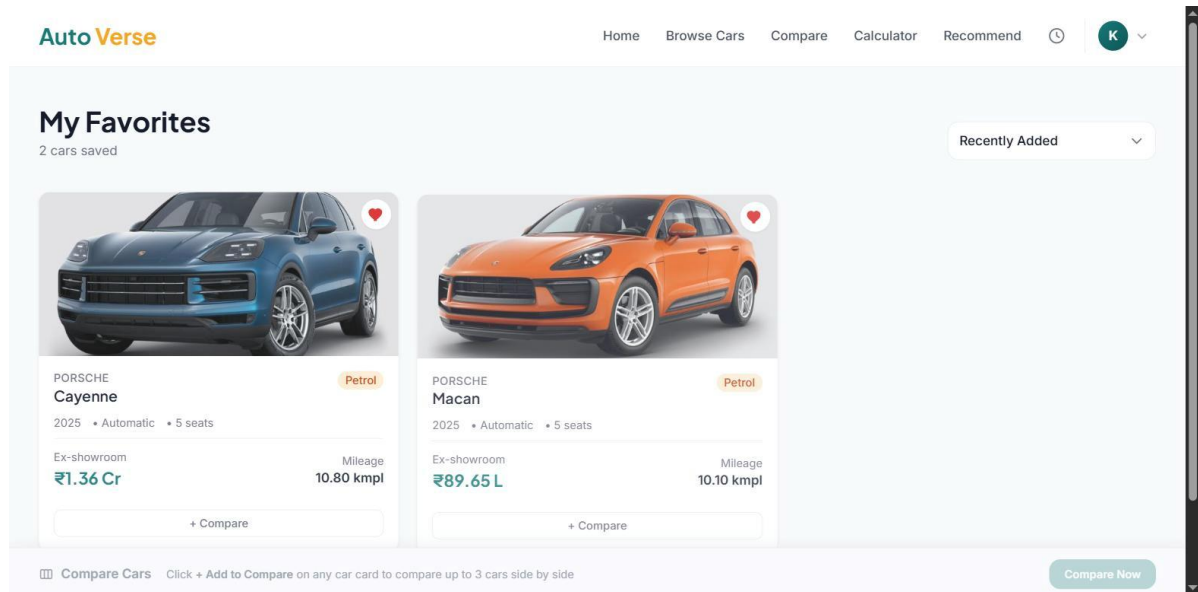


Figure 8.5: Favorites

8.6 Overall System Performance

The platform responds quickly for normal use, with search and filtering staying responsive across the full dataset. The one exception is the very first request after the free-tier backend has been idle for a while, when it can take up to a minute to wake up — this is discussed further in Chapter 9.

CHAPTER 9

LIMITATION AND FUTURE ENHANCEMENT

- **LIMITATION**
- **FUTURE ENHANCEMENT**

CHAPTER 9 LIMITATION AND FUTURE ENHANCEMENT

9.1 Limitation

- AutoVerse is a web application only — there is no dedicated mobile app yet.
- The recommendation engine uses a fixed scoring formula rather than one learned from real user feedback, so its ranking has not been validated against real buyers yet.
- Depreciation figures follow the IRDAI standard, which is a recognized estimate — a car's actual resale price can still vary by city, demand and condition.
- Fuel price is entered manually in the cost calculator rather than fetched automatically for the user's city.
- The free-tier backend hosting goes idle after inactivity, so the very first visit after a quiet period can take up to about a minute to load.
- There is no payment or purchase flow — AutoVerse helps a buyer research and decide, not complete a transaction.

9.2 Future Enhancement

- Build a native mobile app using the existing REST API, without needing backend changes.
- Fetch live, city-wise fuel prices automatically instead of relying on manual entry.
- Learn from user feedback (e.g. marking a recommendation as a good or bad fit) to improve the recommendation ranking over time.
- Add a condition and accident-history questionnaire that adjusts resale value more precisely.
- Add saved searches and comparison sharing for registered users.
- Move the backend to a hosting tier that stays warm, removing the free-tier wake-up delay.

CHAPTER 10

CONCLUSION

▪ CONCLUSION

CHAPTER 10 CONCLUSION

10.1 Conclusion

AutoVerse brings car search, side-by-side comparison, preference-based recommendations and a detailed ownership cost calculator together into a single platform, closing the specific gaps found in existing Indian car-shopping websites during the literature survey in Chapter 2.

The project was built end to end — from a real, researched dataset of 10,000+ Indian cars, through a validated backend API, to a responsive React interface — and is deployed and live for real use rather than existing only as a local demo. Building it also strengthened practical skills in full-stack architecture, database design, and working with real-world data.

While the current version has some known limitations, listed clearly in Chapter 9 rather than left to be discovered later, the platform already delivers on its core goal: giving a car buyer one honest, single place to research a decision that usually takes visiting several different websites.

CHAPTER 11

APPENDICES

- **BUSINESS MODEL**
- **PRODUCT DEPLOYMENT DETAIL**
- **API AND WEB SERVICE DETAILS**

CHAPTER 11 Appendices

11.1 Business Model

AutoVerse is currently built and presented as an academic, portfolio-style project rather than a commercial product. If extended toward a real product, it would naturally fit a service model — offering the platform free to car buyers, with a possible future revenue stream from referral commissions with dealerships, or premium features for dealers (such as priority listing visibility).

11.2 Product Deployment Detail

AutoVerse is deployed using three free-tier cloud services, connected together:

Component	Hosting Service	Role
Frontend	Vercel	Serves the built React application to users.
Backend	Render	Runs the FastAPI application and serves the REST API.
Database	Neon	Hosts the PostgreSQL database (cars, brands, users, favorites).

11.3 API and Web Service Details

AutoVerse exposes its functionality through a RESTful API built with FastAPI. Every endpoint is documented automatically and can be explored interactively at `/docs` on the live backend.

Method	Endpoint	Description
GET	<code>/cars</code>	Paginated, filtered car listing
GET	<code>/cars/{id}</code>	Full car details
GET	<code>/cars/brands</code>	Distinct brand list
GET	<code>/cars/filters/options</code>	All filter dropdown values
GET	<code>/cars/compare?ids=1,2,3</code>	Side-by-side comparison
POST	<code>/cars/recommend</code>	Ranked recommendations from preferences
POST	<code>/calculators/ownership</code>	Ownership cost projection
GET	<code>/calculators/depreciation/{car_id}</code>	Year-by-year depreciation
POST	<code>/auth/register</code>	Create a new user account
POST	<code>/auth/login</code>	Log in and receive an authentication token
GET	<code>/favorites</code>	List the logged-in user's saved cars

Authentication uses JWT (JSON Web Token): once a user logs in, the token is sent with each request to prove who they are, instead of the server having to remember every logged-in session.

BIBLIOGRAPHY

1. Insurance Regulatory and Development Authority of India (IRDAI) — Motor Vehicle Depreciation Schedule, used as the basis for the depreciation feature (Section 7, Section 9).
2. FastAPI Documentation — <https://fastapi.tiangolo.com/>
3. SQLAlchemy Documentation — <https://docs.sqlalchemy.org/>
4. React Documentation — <https://react.dev/>
5. Vite Documentation — <https://vitejs.dev/>
6. Tailwind CSS Documentation — <https://tailwindcss.com/>
7. CarDekho, CarWale, Cars24 — reviewed as existing-system references for the Literature Survey (Chapter 2).